Artificial Intelligence
Introduction and examples

Stefaan Haspeslagh

Vives University of applied sciences – Association KU Leuven – Kortrijk, Belgium

katholieke hogeschool
associatie KU Leuven
vives

Artificial Intelligence: introduction and examples

BLIND SEARCH

katholieke hogeschool
associatie KU Leuven
vives

Blind search

- We have:
  - State-space representation: how to formulate problem, goal, production rules, …
  - From the representation and the rule:
    - → construction of search tree (implicit!)
  - Now we focus on ‘solving the problem’
    - How to ‘search the search tree’
- Credits to prof. Danny De Schreye, KU Leuven

katholieke hogeschool
associatie KU Leuven
vives

Blind search

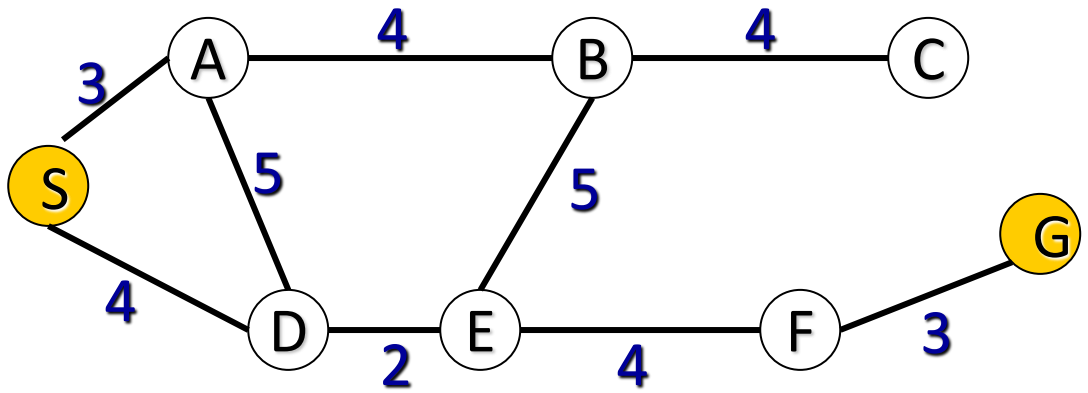
- The algorithm describes how to traverse the search tree

- 2 examples:
  - Breadth-first search (BFS)
  - Depth-first search (DFS)

katholieke hogeschool
associatie KU Leuven
vives

Running example

- We have the following graph:

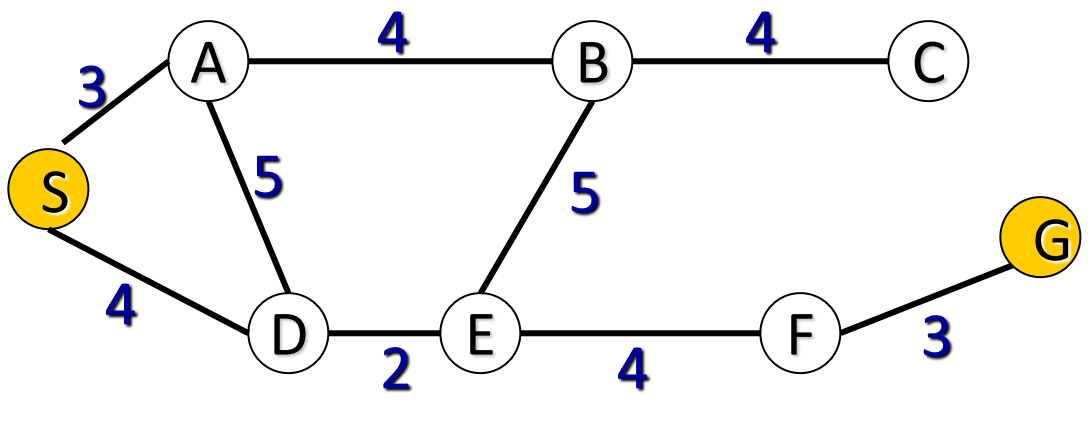


Implicit network

- We want to find a path from vertex S (starting point) to vertex G (the goal)

katholieke hogeschool
associatie KU Leuven
vives

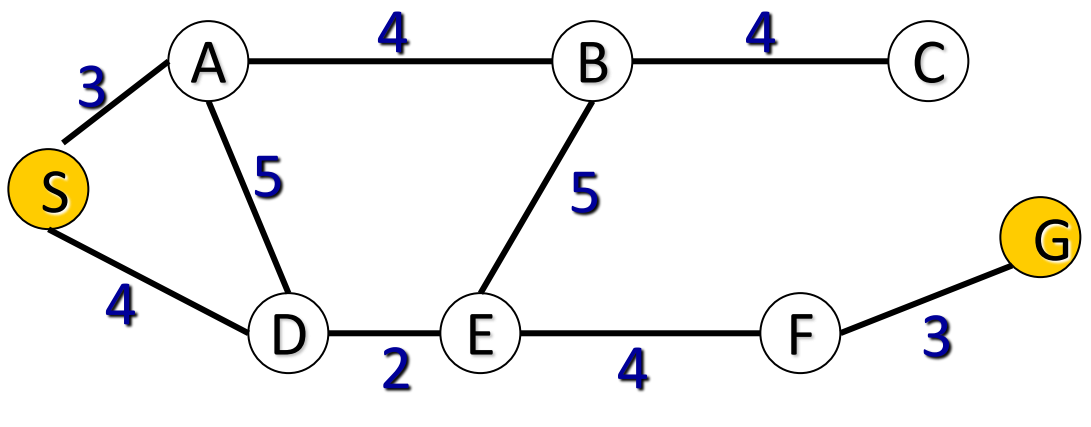
Running example



Implicit network

- Three **possibilities**:
  - We want to find a path → blind search
  - We want to find the optimal path → heuristic search
  - We find the optimal path → optimal search

Running example

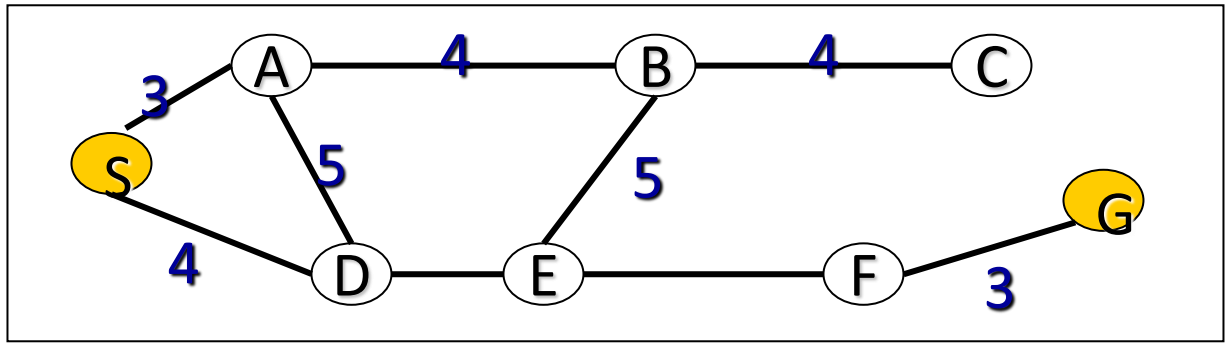


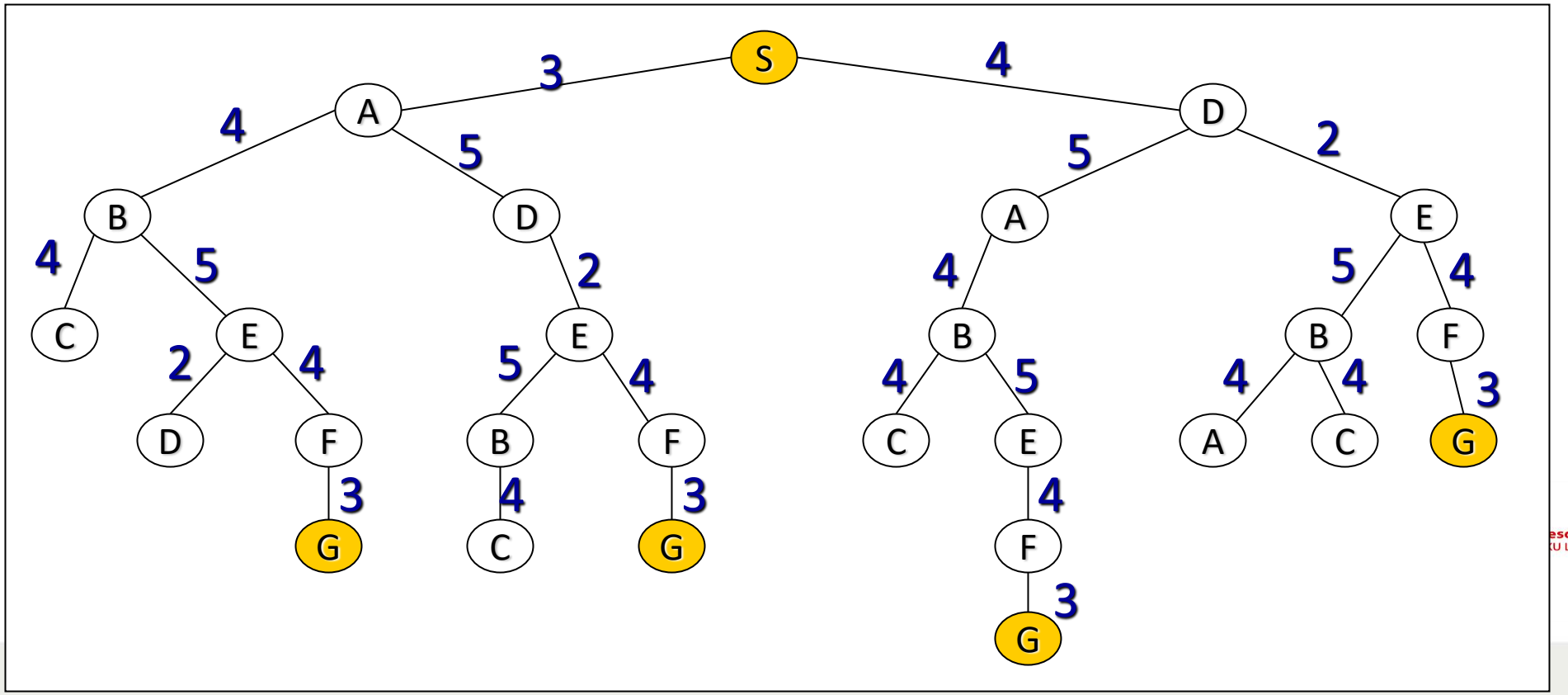
Implicit network

- Two **tasks**:
  - Find a path → computational cost
  - Traverse a path → travel cost

Running example

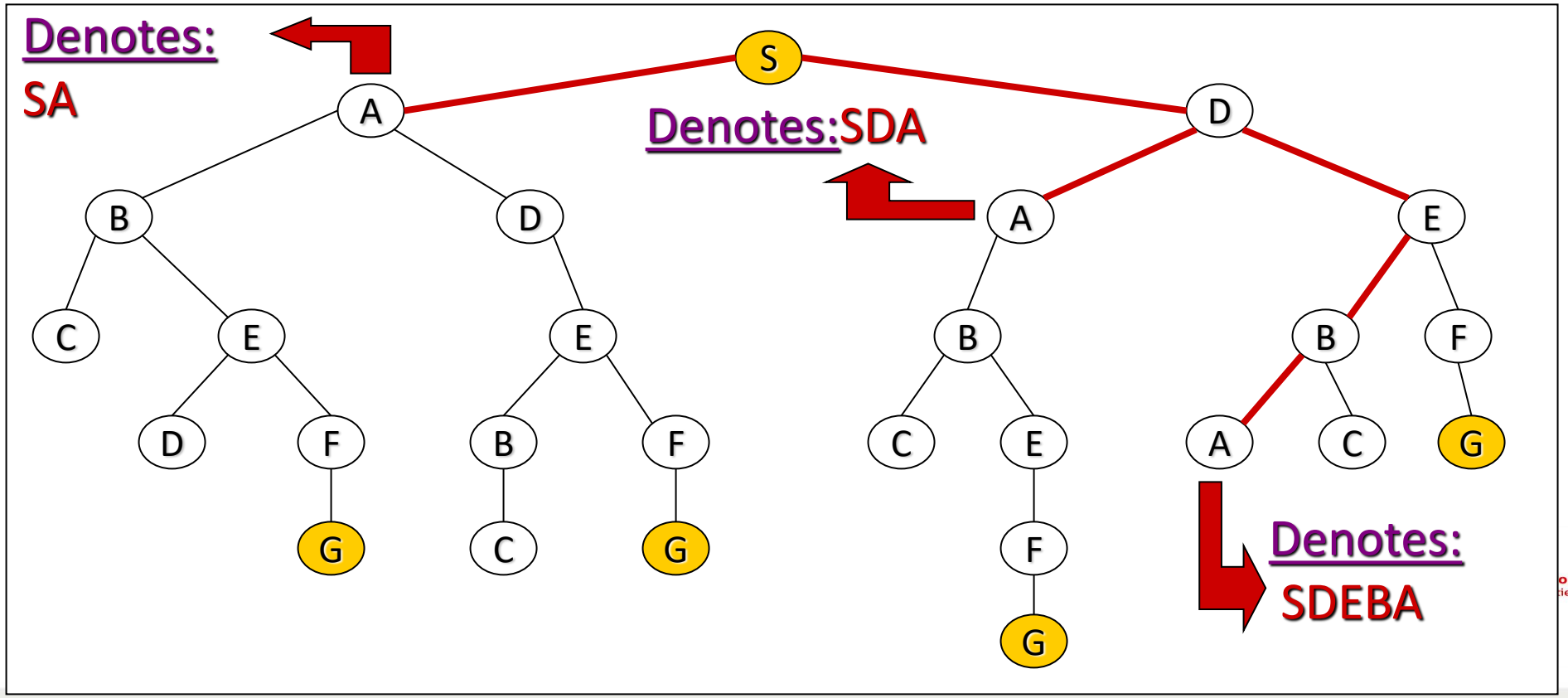
- Graph can be converted to search tree



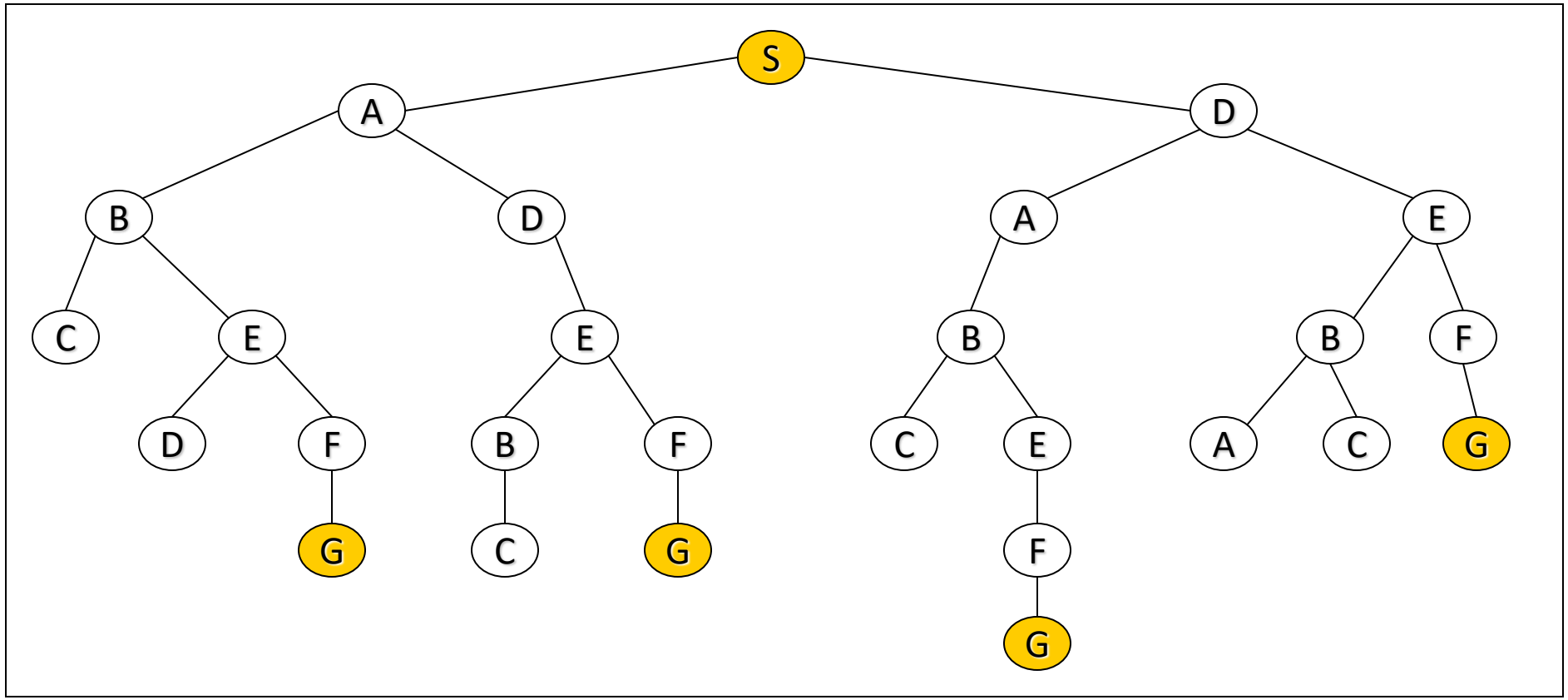


Running example

- Since we are not interested in optimal paths, we can drop the costs
- A node in the tree does not denote itself, but a path from the root to the node



Terminology



- Node, link (or edge), branch
- Parent, child, ancestor, descendant
- Root node, goal node
- Expand / open node / closed node/ branching factor

katholieke hogeschool associatie KU Leuven vives

Artificial Intelligence: introduction and examples

BREADTH-FIRST SEARCH

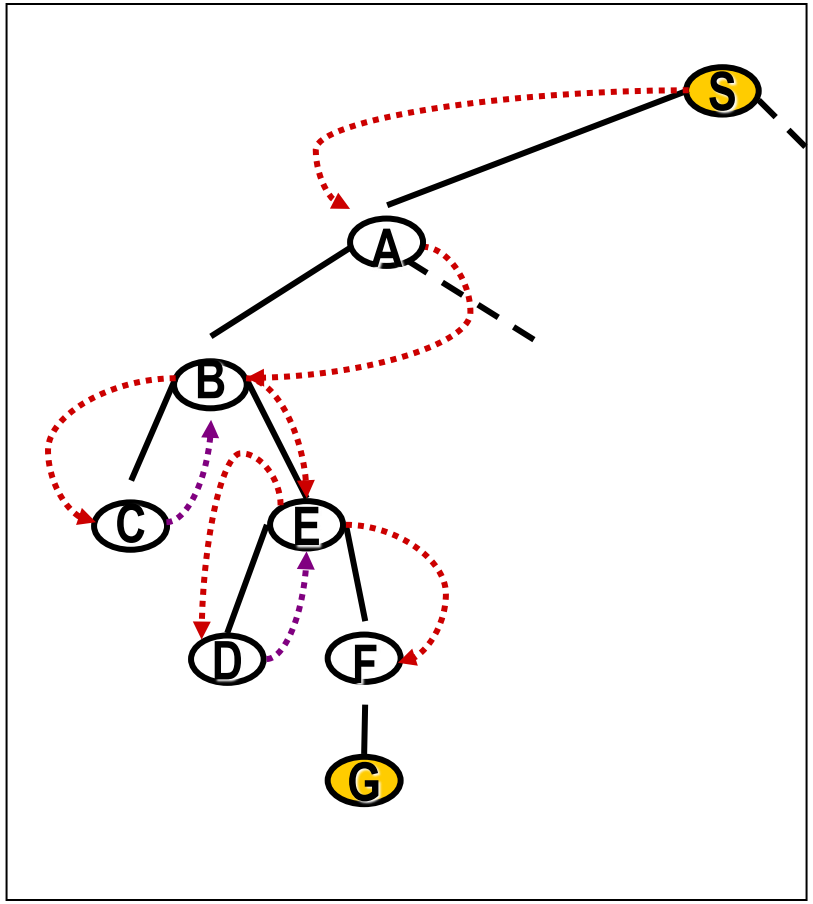
katholieke hogeschool
associatie KU Leuven
vives

Depth-first search

- Expand the tree as deep as possible, returning to upper levels when needed.
  - → backtracking

- *In depth-first search only a few nodes are stored in memory at one time. After the expansion of a node only its successors are saved, and the first successor node is immediately expanded. Thus the search quickly becomes very deep. Only when a node has no successors and the search fails at that depth is the next open node expanded via backtracking to the last branch, and so on.*

Depth-first search



- Start at ‘S’
- Select a child
  - Convention: left-to-right
- Repeatedly go to next child, as long as possible.
- Return to left-over alternatives (higher-up) only when needed.

katholieke hogeschool
associatie KU Leuven
vives

Depth-first search

1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths to the **front** of QUEUE

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

katholieke hogeschool
associatie KU Leuven
vives

Depth-first search on running example

QUEUE	**LOOP**
1. (S)	1. S removed, (SA, SD) computed and added
2. (SA, SD)	2. SA removed, (SAB, SAD, SAS) computed, (SAB, SAD) added
3. (SAB, SAD, SD)	3. SAB removed, (SABA,SABC,SABE) computed, (SABC,SABE) added
4. (SABC, SABE, SAD, SD)	4. SABC removed, (SABCB) computed, nothing added
5. (SABE, SAD, SD)	5. SABE removed, (SABEB,SABED,SABEF) computed, (SABED,SABEF) added
6. (SABED, SABEF, SAD, SD)	6. SABED removed, (SABEDS,SABEDA.SABEDE) computed, nothing added
7. (SABEF, SAD, SD)	7. SABEF removed, (SABEFE,SABEFG) computed, (SABEFG) added
8. (SABEFG, SAD, SD)	8. goal is reached: reports success

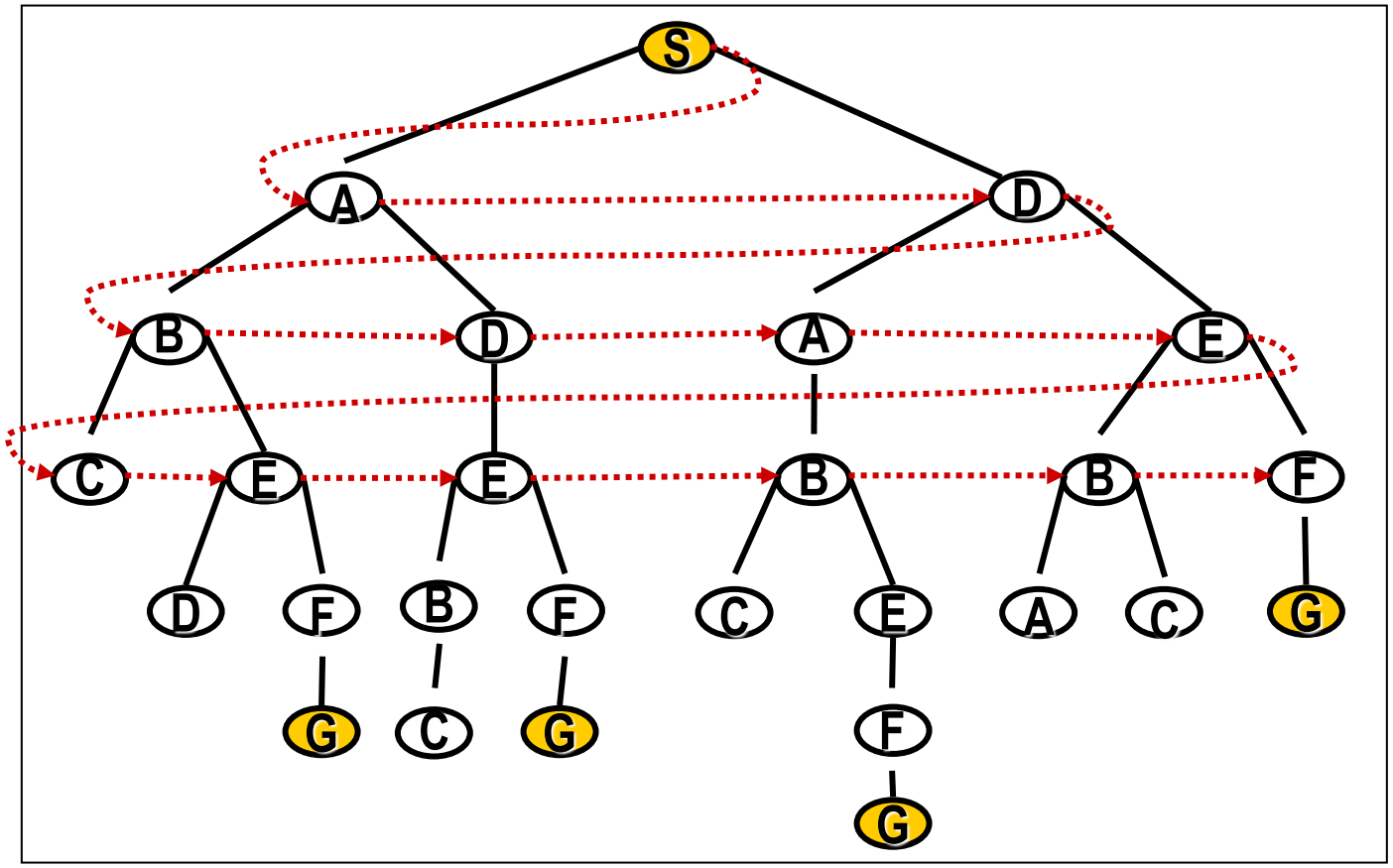
katholieke hogeschool
associatie KU Leuven
vives

Breadth-first search

- Expand the tree layer by layer, progressing in depth.

- *In breadth-first search, the search tree is explored from top to bottom until a solution is found. First every node in the node list is tested for whether it is a goal node, and in the case of success, the program is stopped. Otherwise all successors of the node are generated. The search is then continued on the list of all newly generated nodes. The whole thing repeats until no more successors are generated.*

Breadth-first search



- Start in node S
- Move downwards, level by level, until goal is reached.

katholieke hogeschool
associatie KU Leuven
vives

Breadth-first search

1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE;
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths to the **back** of QUEUE

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

katholieke hogeschool
associatie KU Leuven
vives

Breadth-first search on running example

QUEUE

1. (S)
2. (SA, SD)
3. (SD, SAB, SAD)
4. (SAB, SAD, SDA, SDE)
5. (SAD,SDA,SDE,SABE,SABC)
6. (SABED,SABEF,SADEB,SADEF, SDABC,SDABE,SDEBA,SDEBC, SDEFG)

LOOP

1. S removed, (SA, SD) computed and added
2. SA removed, (SAB, SAD, SAS) computed, (SAB, SAD) added
3. SD removed, (SDA,SDE,SDS) computed, (SDA,SDE) added
4. SAB removed, (SABA,SABE,SABC) computed, (SABE,SABC) added
5. SAD removed, (SADS,SADA,SADE) computed, (SADE) added etc, until QUEUE contains:
6. goal is reached: reports success

katholieke hogeschool
associatie KU Leuven
vives

Depth-first and breadth-first search

- We now have two algorithms to ‘traverse’ the search tree
- What algorithm to choose?
- What algorithm is best?
- Need for criteria to compare algorithms and to select the best suitable
  → See topic 5

Questions

?

Coordinates:

Dr. Stefaan Haspeslagh

A.I. Research & Education

Vives University of applied sciences – KU Leuven Association

E-mail: stefaan.haspeslagh@vives.be

katholieke hogeschool
associatie KU Leuven
vives